09 · Component Communication (input, output, model)

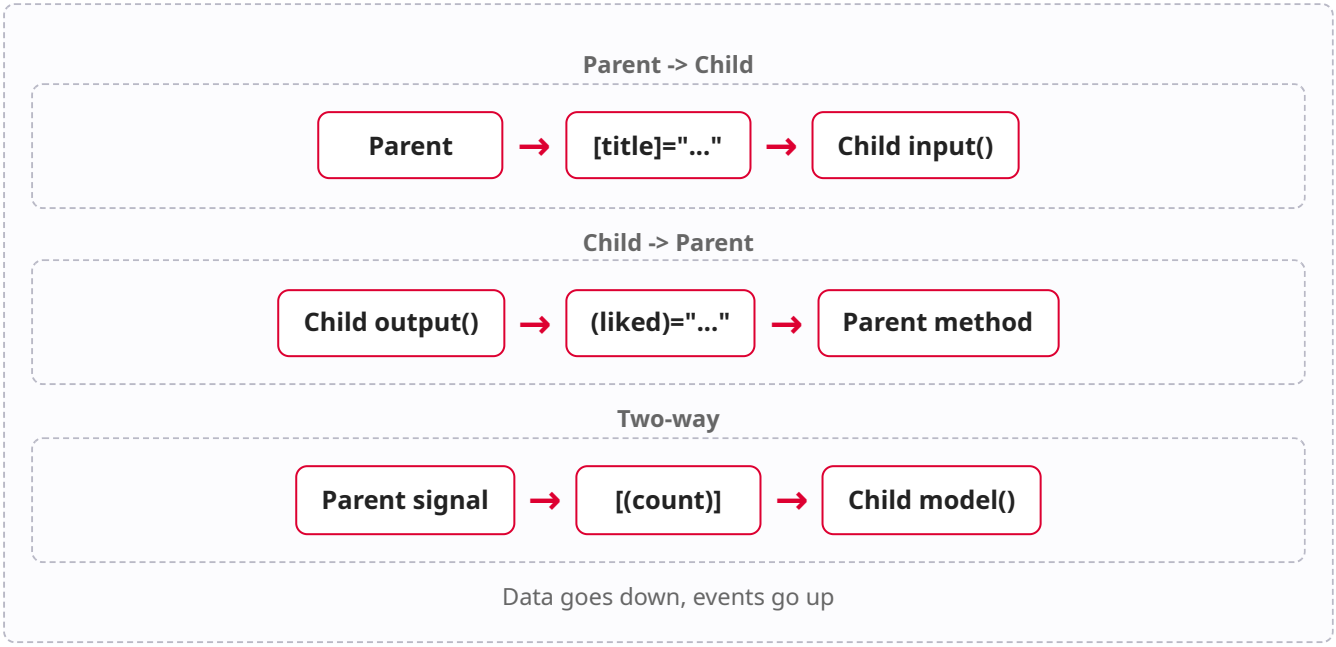
Beginner Angular learning notes

a. Definition

Components rarely work alone. A **parent** passes data to a **child** using an **input** and the child tells the parent something happened using an **output** event. The rule of thumb is: *data goes down, events go up*.

Modern Angular uses the functions `input()`, `input.required()` and `output()`. `model()` creates a writable input that also emits changes, so the parent can use two-way binding `[(name)]`.

b. Diagram



c. Syntax

```
// child
title  = input.required<string>();   // value from parent
liked  = output<string>();           // event to parent
count  = model(0);                   // two-way value

// parent template
<app-child [title]="x" (liked)="onLiked($event)" [(count)]="total" />
```

d. Example Program

`Programs/09_Component_Communication/like-button.component.ts`

```
import { Component, input, model, output, signal } from '@angular/core';
```

```ts
// ---- Child 1: input() and output() ----
@Component({
  selector: 'app-like-button',
  template: `<button (click)="liked.emit(title())">Like "{{ title() }}"</button>`,
})
export class LikeButtonComponent {
  title = input.required<string>(); // data coming IN from the parent
  liked = output<string>();         // events going OUT to the parent
}

// ---- Child 2: model() = two-way binding ----
@Component({
  selector: 'app-stepper',
  template: `
    <button class="minus" (click)="count.set(count() - 1)">-</button>
    {{ count() }}
    <button class="plus" (click)="count.set(count() + 1)">+</button>
  `,
})
export class StepperComponent {
  count = model(0);
}

// ---- Parent ----
@Component({
  selector: 'app-root',
  imports: [LikeButtonComponent, StepperComponent],
  template: `
    <app-like-button title="Angular Basics" (liked)="onLiked($event)" />
    <p>Last liked: {{ last() || 'nothing yet' }}</p>

    <app-stepper [(count)]="total" />
    <p>Parent sees total = {{ total() }}</p>
  `,
})
export class AppComponent {
  last = signal('');
  total = signal(0);

  onLiked(title: string) {
    this.last.set(title);
  }
}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Click the **Like** button.
2. Click the stepper's **+** twice.

```
Like "Angular Basics"
Last liked: Angular Basics
- 2 +
Parent sees total = 2
```

f. Real-World Example (English + தமிழ்)

English

A mother (parent) asks a child to bring half a kilo of rice (input); the child comes back and shouts 'got it!' (output). Data flows down, events flow up.

தமிழ்

ஒரு வீட்டில் அம்மா (parent) சமையலறையில் இருக்கும் குழந்தைக்கு (child) 'அரை கிலோ அரிசி எடுத்து வா' என்று சொல்கிறார் - இது **input** (மேலிருந்து கீழே தகவல்). குழந்தை அரிசியுடன் வந்து 'எடுத்துவிட்டேன்!' என்று குரல் கொடுக்கிறது - இது **output** (கீழிருந்து மேலே நிகழ்வு).

இருவரும் ஒரே எண்ணை (எ.கா. மீதமுள்ள அரிசி அளவு) பகிர்ந்து புதுப்பித்துக்கொண்டால் அது **model()** - இருவழி இணைப்பு.

g. Pros

- Clear one-way data flow
- Type-safe inputs/outputs
- Required inputs checked at compile time

h. Cons

- Deep trees cause prop drilling (use services)
- Overusing two-way binding hides data flow

i. Advanced / New Features

- Function-based `input()`, `output()`, `model()` (stable v19)
- Old `@Input/@Output` still supported

j. Common Mistakes

- Trying to change an input's value inside the child; inputs are read-only (use `model()` if the child must write).
- Forgetting `()` when reading a signal input: `title` vs `title()`.
- Using `$event` wrongly; for `output<string>()`, `$event` is the emitted string.
- Passing data up by changing the parent's object from the child (hard-to-trace bugs). Emit an event instead.

k. Interview Questions

Q1. How does a parent pass data to a child?
With property binding to an `input()` of the child, e.g. `<app-child [title]="t" />`.

Q2. How does a child notify its parent?
By emitting an `output()` event that the parent handles with `(eventName)="handler($event)"`.

Q3. What is `input.required`?
An input that the parent must provide; the compiler errors if it is missing.

Q4. What is `model()`?
A signal that works as an input and an output at once, enabling `[(prop)]` two-way binding.

Q5. Old vs new syntax?
Older code uses `@Input()` / `@Output() EventEmitter` decorators; both still work, but functions are recommended.

l. Practice Exercises

1. Create a `rating-star` child that takes a `value` input and emits a `rated` event when a star is clicked.
2. Make a `todo-item` child that receives a task object and emits `delete` with the task id.
3. Use `model()` to build a `toggle-switch` that parents can bind with `[(on)]`.